



Trabajo de Fin de Grado

Grado Superior Doble en Desarrollo de Aplicaciones Web y Multiplataforma

JHTTP

Desarrollo de un servidor HTTP/HTTPS en Java.

Autor: *Sergio de Santiago Redondo*

Tutor: *Daniel Sierra Solis*

Curso: *2025/2026*

Í N D I C E

INTRODUCCIÓN

1.1. Descripción.....	2
-----------------------	---

ESTRUCTURA DEL PROYECTO

2.1. Descripción.....	3
-----------------------	---

ESTADO ACTUAL DEL PROYECTO

3.1. Arranque del programa.....	4
---------------------------------	---

3.2. Lectura del mensaje entrante.....	5
--	---

3.3. Gestión de la respuesta saliente.....	7
--	---

ALOJAMIENTO DEL CÓDIGO

4.1. Gestión de los repositorios.....	8
---------------------------------------	---

4.2. Repositorio de código.....	8
---------------------------------	---

4.3. Repositorio de entrega.....	8
----------------------------------	---

CAPÍTULO 1

INTRODUCCIÓN

1.1. Descripción

El presente documento tiene como objetivo mostrar el progreso en el desarrollo del Trabajo de Fin de Grado JHTTP.

CAPÍTULO 2

ESTRUCTURA DEL PROYECTO

2.1. Descripción

La aplicación está dividida, por el momento, en 6 paquetes diferenciados, cada uno con su respectiva función

- **core:** Alberga el núcleo de la aplicación. En él, residen las clases **Acceptor**, la cual gestiona las conexiones entrantes; **Worker**, que maneja la petición y respuesta a la conexión; y **Dispatcher**, que actúa como intermediario entre las clases *Acceptor* y *Worker*, permitiendo de esta manera habilitar más o menos hilos *Worker* en base a las necesidades de uso.
- **http:** Gestiona el procesamiento de los mensajes HTTP. Contiene las clases **HttpRequestParser**, para gestionar los mensajes entrantes; y **HttpResponseBuilder**, cuyo objetivo es el de proporcionar una interfaz sencilla para construir los mensajes salientes.
- **module:** Gestiona los módulos personalizables de la aplicación. Aquí residen las clases encargadas de dar soporte al contenido dinámico. Estos son cargados en el arranque de la aplicación. **Este paquete no está desarrollado.**
- **security:** Contiene las clases encargadas de asegurar que la aplicación sea más segura y robusta, buscando así evitar el cuelgue de la aplicación. **Este paquete no está desarrollado.**
- **config:** Administra los procesos necesarios para el correcto arranque y funcionamiento del programa. Este paquete contiene las clases **ConfigLoader**, el cual permite utilizar una configuración personalizada o, en su defecto, la configuración predeterminada; **ServerConfig**, que contiene la configuración del servidor; y **ServerBootstrap**, sirviendo esta como punto de acceso a la aplicación.
- **util:** Utilidades que pueden ser utilizadas en cualquier contexto de la aplicación, como gestión más compleja de strings.

CAPÍTULO 3

ESTADO ACTUAL DEL PROYECTO

3.1. Arranque del programa

Se gestiona el uso del puerto indicado por el usuario, en el ejemplo el 8080, se configura la gestión por parte del socket del servidor como no bloqueante y se arrancan los componentes principales, Acceptor, Dispatcher y su correspondiente pool de Workers. Hecho esto, se da inicio al funcionamiento del servidor.

```
public void start() throws Exception { 1 usage  Sergio
    serverSocket = ServerSocketChannel.open();

    final var port = new InetSocketAddress(config.getPort());
    serverSocket.bind(port);
    log.info( message: "Server socket bound to port '{}'", serverSocket.socket().getLocalPort());
    serverSocket.configureBlocking(false);

    final var selector = Selector.open();
    serverSocket.register(selector, SelectionKey.OP_ACCEPT);

    final var workers = new Worker[]{
        new Worker( name: "a"),
        new Worker( name: "b"),
        new Worker( name: "c"),
        new Worker( name: "d")
    };
    final var dispatcher = new Dispatcher(workers);
    dispatcher.start();

    final var acceptor = new Acceptor(serverSocket, selector, dispatcher);
    final var acceptorThread = new Thread(acceptor, name: "acceptor");

    acceptorThread.setDaemon(false);
    acceptorThread.start();
}
```

Sirvan los logs como muestra del que el proceso se lleva a cabo con éxito.

```
2026-05-08 18:54:48.211 INFO --- [main ] s.j.config.ServerBootstrap - Server socket bound to port '8080'
2026-05-08 18:54:48.304 INFO --- [worker-2] s.j.core.Worker - [b] worker started
2026-05-08 18:54:48.304 INFO --- [worker-3] s.j.core.Worker - [c] worker started
2026-05-08 18:54:48.304 INFO --- [worker-1] s.j.core.Worker - [a] worker started
2026-05-08 18:54:48.305 INFO --- [worker-4] s.j.core.Worker - [d] worker started
2026-05-08 18:54:48.306 INFO --- [acceptor] s.j.core.Acceptor - Server running
```

3.2. Lectura del mensaje entrante

Una vez aceptada la petición por el Acceptor, el resto de la gestión es mandado a uno de los Workers del Dispatcher, estos corren de manera permanente durante todo el funcionamiento de la aplicación.

```
@Override Sergio
public void run() {
    log.info("Server running");

    while (running) {
        try {
            selector.select();
        } catch (final IOException _) {}

        final var keys = selector.selectedKeys();
        for (final var key : keys) {
            try {
                if (!key.isValid()) continue;

                if (key.isAcceptable()) {
                    accept(key);
                }
            } catch (final Exception _) {}
        }
        keys.clear();
    }
}
```

```
private void accept(final @NonNull SelectionKey key) throws IOException {
    final var server = (ServerSocketChannel) key.channel();
    final var client = server.accept();

    if (client == null) return;

    client.configureBlocking(false);
    dispatcher.dispatch(client);
}
```

```
public void dispatch(final @NonNull SocketChannel channel) { 1 usage Sergio
    final var index = Math.abs(counter.getAndIncrement() % workers.length);
    workers[index].register(channel);
}
```

El Worker recibe la orden de trabajo y registra una clave en modo lectura. Cuando el tick se encuentra en dicho modo, se procesa la petición entrante y se genera una respuesta. En este caso, la respuesta generada es un 200 OK genérico. Una vez hecho esto, se añade el modo escritura a dicha clave.

```
private void read(final @NonNull SelectionKey key) throws IOException { 1 usage  Sergio *
    final var channel = (SocketChannel) key.channel();
    final var buffer = ByteBuffer.allocate(capacity: 4096);

    final var bytesRead = channel.read(buffer);

    if (bytesRead == -1) {
        closeKey(key);
        return;
    } else if (bytesRead == 0) {
        return;
    }

    buffer.flip();
    touchActivity(key);

    key.attach(
        HttpResponseBuilder.ok( body: "<!DOCTYPE html>"
            + "<html><head lang=es><title>JHTTP</title><meta charset=\"UTF-8\"></head>"
            + "<body>Hello World!</body>"
            + "</html>")
    );

    key.interestOps(SelectionKey.OP_WRITE);
    log.debug(message: "[{}] request received ({} bytes), switching to WRITE", name, bytesRead);
}
```

3.3. Gestión de la respuesta saliente

Cuando el tick está en modo lectura, se comienza a escribir en el socket del cliente. Una vez terminado, se elimina dicha clave y se da por cerrada finalizado el intercambio de mensajes. Entre las posibles cabeceras, o headers, que se pueden recibir en el mensaje entrante, existe la cabecera *Connection*, con valor posible *keep-alive*. Esto indica si la petición se ha llegado a completar por parte del cliente. Este comportamiento no está aún implementado.

```
private void write(final @NonNull SelectionKey key) throws IOException {
    final var channel = (SocketChannel) key.channel();
    final var response = (ByteBuffer) key.attachment();

    channel.write(response);

    if (response.hasRemaining()) {
        return;
    }

    closeKey(key);
    log.debug( message: "[{}] response sent", name);
}
```

The screenshot shows a web browser window with the address bar displaying 'http://localhost:8080'. The page content is 'Hello World!'. Below the page, the Chrome DevTools network tab is open, showing a table of network requests. The table has columns for 'Estado', 'Método', 'Dominio', and 'Archivo'. Two requests are listed, both with a status of 200, method GET, and domain localhost:8080. The first request is for the root path '/', and the second is for 'favicon.ico'.

Estado	Método	Dominio	Archivo
200	GET	localhost:8080	/
200	GET	localhost:8080	favicon.ico

Sirva el siguiente log como muestra de que se realiza la petición por dos canales distintos pese a que el cliente indica que la petición se puede mantener abierta.

```
2026-05-08 19:21:34.237 DEBUG --- [worker-1] s.j.core.Worker - [a] registered channel java.nio.channels.SocketChannel[connected local=/127.0.0.1:8080 remote=/127.0.0.1:54245]
2026-05-08 19:21:34.245 DEBUG --- [worker-1] s.j.core.Worker - [a] request received (552 bytes), switching to WRITE
2026-05-08 19:21:34.247 DEBUG --- [worker-1] s.j.core.Worker - [a] response sent
2026-05-08 19:21:34.278 DEBUG --- [worker-2] s.j.core.Worker - [b] registered channel java.nio.channels.SocketChannel[connected local=/127.0.0.1:8080 remote=/127.0.0.1:54246]
2026-05-08 19:21:34.278 DEBUG --- [worker-2] s.j.core.Worker - [b] request received (572 bytes), switching to WRITE
2026-05-08 19:21:34.278 DEBUG --- [worker-2] s.j.core.Worker - [b] response sent
```

The screenshot shows the 'Cabeceras de la petición (552 B)' section of the Chrome DevTools network tab. The 'Sin procesar' toggle is turned on. The headers listed are:

- Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
- Accept-Encoding: gzip, deflate, br, zstd
- Accept-Language: es-ES;q=0.9,en-US;q=0.8,en;q=0.7
- Cache-Control: no-cache
- Connection: keep-alive
- Cookie: Idea-2dc90a3=9734c99b-c860-4580-9a1b-9dd866089216

CAPÍTULO 4

ALOJAMIENTO DEL CÓDIGO

4.1. Gestión de los repositorios

Puesto que se plantea seguir desarrollando la aplicación una vez terminado el plazo de entrega, se ha decidido gestionar el código en dos repositorios.

4.2. Repositorio de código

<https://github.com/sdevsantiago/jhttp>

El código se subirá a este repositorio. La documentación presente en el mismo será sobre el mismo y no incluirá documentación que haga referencia al Trabajo de Fin de Grado.

4.3. Repositorio de entrega

<https://github.com/sdevsantiago/2026-tfg-dam>

La documentación referente al Trabajo de Fin de Grado se subirá exclusivamente a este repositorio, siendo este el repositorio que se entregará como evaluación final. La actualización del código está gestionada de manera automática y no requiere intervención, salvo en caso de fallo. Las actualizaciones de este repositorio cesarán una vez se valide la entrega final del trabajo.